Interconnection Structures

Condensed Study Notes

Generated: 2026-05-24 23:47 UTC

COMPUTER ARCHITECTURE & ORGANIZATION LECTURE (WEEK 2

This lecture introduces fundamental concepts in computer architecture and organization. The focus is on understanding how computer systems are structured and how they operate at a foundational level.

Key areas covered include:

- **Computer System Components:** The basic building blocks of a computer system, including the CPU, memory, and input/output devices.
- **CPU Functionality:** How the Central Processing Unit (CPU) executes instructions, typically involving a fetch-decode-execute cycle.
- **Performance Metrics:** Understanding factors that influence system performance, such as clock speed, but also recognizing that bottlenecks can occur elsewhere in the system.
- **Parallelism:** The concept of performing multiple tasks simultaneously, often facilitated by multiple CPU cores.

Interconnection Structures

Computers consist of three fundamental modules: the Processor, Memory, and Input/Output (I/O) devices.

These modules need to communicate. The paths that connect them for communication are collectively known as the interconnection structure.

The design of this structure is determined by the types of data exchanges required between the modules.

I/O module

From an internal perspective, the I/O module functions similarly to memory.

An I/O module handles communication between external devices and the rest of the computer system. It can control multiple external devices, each accessible via a unique address called a port.

Key Transfers Supported by the Interconnection Structure:

- **Memory to Processor:** The processor retrieves instructions or data from memory.
- **I/O to Processor:** The processor reads data from an external device through the I/O module.
- **Processor to I/O:** The processor sends data to an external device.
- **I/O to or from Memory:** The I/O module can directly exchange data with memory without involving the processor. This is known as **Direct Memory Access (DMA)**.

Additionally, the I/O module can send interrupt signals to the processor to signal events.

Operations are categorized as read or write.

Memory

Memory stores data and instructions. Each location within memory is identified by a unique numerical address, ranging from 0 to N-1. A memory module typically contains N words, all of the same length. Operations on memory involve either reading data from a specified address or writing data to a specified address, controlled by read/write signals.

Complex Instruction Set Architecture (CISC

CISC aims to increase CPU performance by creating complex hardware to handle complex low-level language statements. A single CISC instruction can perform multiple operations, such as loading data, performing an evaluation (like multiplication), and storing the result, all within one command. This contrasts with architectures where these steps would require separate instructions.

Instruction set Architecture: CISC, RISC

Instruction Set Architecture (ISA) defines the set of commands a processor understands. Two major approaches exist: CISC and RISC.

CISC (Complex Instruction Set Architecture):

- A single instruction can perform multiple low-level operations.
- Example: A CISC instruction might load data from memory, perform an arithmetic operation, and store the result back, all in one step.
- This complexity aims to reduce the number of instructions needed for a task.

RISC (Reduced Instruction Set Architecture):

- Focuses on a smaller set of simple, highly optimized instructions.
- Each instruction typically performs a single, basic operation (e.g., load, store, add).
- Complex operations are achieved by combining multiple simple RISC instructions.
- Analogy: Think of CISC as a multi-tool that can do many things but might be bulky, while RISC is a set of individual, sharp tools, each perfect for one job.

Reduced Instruction Set Architecture (RISC

RISC aims for simpler hardware by using an instruction set focused on basic operations. These instructions handle fundamental steps like loading data, performing evaluations, and storing results, with each instruction performing a single, distinct task. For example, a load command specifically fetches data, and a store command specifically saves it.

The “best” way to design a CPU has been a subject of debate

The optimal CPU design has been debated, centering on the complexity and number of instructions.

Two primary approaches exist:

- **CISC (Complex Instruction Set Computer):** Uses a large set of complex machine language instructions. A single CISC instruction can perform multiple operations, like loading data, performing a calculation, and storing the result.
- **RISC (Reduced Instruction Set Computer):** Uses a smaller, simpler set of instructions. Each RISC instruction typically performs a single operation (e.g., load, evaluate, or store), requiring more individual instructions to complete a complex task but potentially using fewer cycles per instruction.

Characteristic of RISC

RISC (Reduced Instruction Set Computer) architectures are characterized by several key features that simplify processor design and execution:

- **Simpler Instructions:** Instructions are basic and easy to decode.
- **Single-Word Instructions:** Each instruction fits within a single memory word, streamlining fetching.
- **Single-Cycle Execution:** Most instructions complete execution within a single clock cycle.
- **More Registers:** A larger number of general-purpose registers are available.
- **Simple Addressing:** Uses straightforward methods to access memory locations.
- **Fewer Data Types:** Supports a limited set of data formats.
- **Pipelining Enabled:** The simple, uniform instruction format facilitates instruction pipelining, allowing multiple instructions to be in different stages of execution simultaneously.

Characteristic of CISC

CISC (Complex Instruction Set Architecture) is characterized by several key features:

1. **Complex Instructions:** CISC uses instructions that can perform multiple operations in a single step (e.g., loading data from memory, performing an arithmetic operation, and storing the result). This complexity leads to more intricate instruction decoding.

2. **Larger Instruction Size:** Instructions in CISC can be larger than a single memory word, meaning they might require multiple memory fetches to be fully loaded.

3. **Multi-Cycle Execution:** Due to their complexity, CISC instructions often take more than one clock cycle to complete.

4. **Fewer General-Purpose Registers:** Operations can often be performed directly on data in memory, reducing the reliance on a large number of general-purpose registers.

5. **Complex Addressing Modes:** CISC supports a variety of ways to specify memory locations for instructions, making memory access more flexible but also more complex.

6. **More Data Types:** CISC architectures typically support a wider range of data types directly.

In contrast to RISC, where an operation like adding two numbers would involve separate load, operate, and store instructions, CISC uses a single `ADD` instruction to achieve the same result. While CISC programs can be shorter and require less memory, they demand more complex hardware to decode and execute these powerful instructions.

CISC vs. RISC: Which is Better?

The debate on whether CISC or RISC is 'better' has no single correct answer, as both have advantages and disadvantages.

- **RISC** aims to reduce the number of clock cycles per instruction, but this may increase the total number of instructions needed for a program.
- **CISC** aims to minimize the number of instructions per program, but this can lead to an increase in cycles per instruction.

Historically, CISC evolved from the need to simplify assembly programming. With the rise of high-level languages, the reliance on complex assembly instructions decreased, favoring the simpler RISC approach.

Examples:

- **CISC:** x86 CPUs
- **RISC:** ARM, PowerPC, Sparc

Difference

This section highlights key distinctions between two CPU design philosophies, focusing on their hardware and software characteristics.

Focus:

- One approach prioritizes software complexity, while the other focuses on hardware simplicity.

Registers:

- The software-focused approach requires a larger number of registers, whereas the hardware-focused approach needs fewer.

Operations:

- The hardware-focused approach can perform arithmetic operations directly between memory and registers (REG to REG, REG to MEM, or MEM to MEM).
- The software-focused approach is limited to register-to-register arithmetic operations.

Instruction Characteristics:

- Instructions in the software-focused approach have variable sizes.
- Instructions in the hardware-focused approach are fixed in size.
- The software-focused approach uses transistors primarily for more registers.
- The hardware-focused approach uses transistors for storing complex instructions.
- Instructions in the software-focused approach result in a larger overall code size.
- Instructions in the hardware-focused approach result in a smaller code size.
- An instruction in the software-focused approach typically takes more than one clock cycle to execute.
- An instruction in the hardware-focused approach can be executed in a single clock cycle.

Instruction and Instruction Types

Beyond the RISC/CISC distinction, computer instruction sets can be classified by several characteristics. The instruction set acts as the interface between software and hardware, defining the actions the hardware will perform.

Key characteristics of instructions include:

- **Addressing modes**: How operands (the data an instruction operates on) are specified.
- **Numbers of operands**: The quantity of operands an instruction can handle.
- **Types of operations supported**: The range of actions an instruction can perform.
- **Instruction length**: Whether instructions have a fixed or variable size.

Word length

Processors can be characterized by their "word length" (e.g., 4-bit, 8-bit, 16-bit, 32-bit). This refers to the typical size of data units the processor handles.

In some architectures, the length of a data word, an instruction, and an address are identical. However, for processors designed for smaller data words, instructions and addresses might be longer than the basic data word.

Note: The way data is ordered in memory (little-endian vs. big-endian) is a related concept to explore.

INSTRUCTION SEQUENCING

Programs are executed by processing instructions in a specific order. This order is typically determined by a Program Counter (PC), which holds the address of the next instruction. The processor fetches and executes instructions sequentially, moving from one address to the next.

Instruction Execution Cycle:

This process involves two main phases:

1. **Instruction Fetch:** The instruction located at the memory address stored in the PC is retrieved and loaded into the processor's Instruction Register (IR).

2. **Instruction Execute:** The instruction in the IR is analyzed to determine the required operation and then performed.

This sequential processing, guided by the PC, is known as **straight-line sequencing**.

Basic Instruction Types:

Instructions can be categorized by their operations:

1. **Data Transfer:** Moving data between memory and processor registers.

2. **Arithmetic & Logic Operations:** Performing calculations and logical comparisons on data.

3. **Program Sequencing & Control:** Altering the normal flow of instruction execution (e.g., jumps, branches).

4. **I/O Transfers:** Communicating with input/output devices.

Instruction Formats (Operand Addressing):

Instructions can be classified by how they specify operands:

- **Zero Address Instructions:** Operands are implicitly managed, often using a structure like a push-down stack.
- **One Address Instructions:** Use an accumulator register for operations. Example: "Add contents of A to accumulator & store sum back to accumulator."
- **Two Address Instructions:** Allow two operands to be specified. Example: "Adding contents of R1, R2 & place sum in R3."
- **Three Address Instructions:** Specify three operands. Example: Equivalent to `B <- [A] + [B]`.

Register Transfer Notation (RTN):

- Used to describe operations at a high level.
- Left-hand side: Name of a location (e.g., a register or memory address).
- Right-hand side: Denotes a value.